AI ASSISTED CODING
ASSIGNMENT - 21.1
ROLL NO. 2303A510C8
B-27

Task 1 – AI-Assisted Pair Programming

- Use AI to generate an initial program (e.g., a simple To-Do List Manager).
- Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).
- Compare the original and improved versions to understand the role of AI in collaboration.

Prompt:

Generate a simple Python To-Do List Manager with basic features: add task, view tasks, and delete task. Keep it very simple without advanced error handling or optimizations.

Code:

```
tasks = []

def add_task(task_name):
    tasks.append(task_name)
    print(f"Task '{task_name}' added.")

def view_tasks():
    if not tasks:
        print("No tasks in the list.")
    else:
print("\n--- Your To-Do List ---") for i,
        task in enumerate(tasks):
            print(f"{i + 1}. {task}")
        print("  ")
```

```
def delete_task(task_number):
    if 1 <= task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
        print(f"Task '{removed_task}' deleted.")
    else:
        print("Invalid task number.")

def run_todo_list_manager():
    while True:
        print("\nTo-Do List Manager")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Delete Task")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            task = input("Enter the task to add: ")
            add_task(task)
        elif choice == '2':
            view_tasks()
        elif choice == '3':
            view_tasks()
            try:
                task_num = int(input("Enter the number of the task to
delete: "))
```

d

e

l

e

t

e

_

t

a

s

k

(

t

a

s

k

_

n

```
u                                       l
m                                       u
)                                       e
e                                       E
x                                       r
c                                       r
e                                       o
p                                       r
t                                       :
V                                           print("Invalid input. Please enter
a                                           a number.")
        elif choice == '4':
            print("Exiting To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

# Run the To-Do List Manager
run_todo_list_manager()
```

Output:

```
To-Do List Manager
  Add Task
  View Tasks
  Delete Task
  Exit
Enter your choice: 1
Enter the task to add: Grocery
Task 'Grocery' added.

To-Do List Manager
  Add Task
  View Tasks
  Delete Task
  Exit
Enter your choice: 2

====Your To-Do List====
  Grocery


To-Do List Manager
  Add Task
  View Tasks
  Delete Task
```

Prompt:

Improve the To-Do List Manager you created by adding enhancements such as:

- input validation
- error handling
- saving/loading tasks to a file
- better structure or refactoring
- additional features like marking tasks as complete

Provide the improved code.

Enhanced code:

```
import json
import os

FILENAME = 'tasks.json'
tasks = []

def load_tasks(filename=FILENAME):
    global tasks
    if os.path.exists(filename):
        try:
            with open(filename, 'r') as f:
                tasks = json.load(f)
            print(f"Tasks loaded from {filename}.")
        except json.JSONDecodeError:
            print(f"Error: Corrupt or empty {filename}. Starting with an
empty list.")
tasks = []
except Exception as e:
            print(f"An error occurred while loading tasks: {e}. Starting
with an empty list.")
tasks = []
else:
print("No existing task file found. Starting with an empty list.")

def save_tasks(filename=FILENAME):
    try:
        with open(filename, 'w') as f:
            json.dump(tasks, f, indent=4)
        print(f"Tasks saved to {filename}.")
    except Exception as e:
print(f"An error occurred while saving tasks: {e}")

def add_task(task_name):
if not task_name.strip():
```

```
        return
    tasks.append({'name': task_name.strip(), 'completed': False})
    print(f"Task '{task_name.strip()}' added.")
    save_tasks()

def view_tasks():
    if not
    tasks:
        print("No tasks in the list.")
    else:
        print("\n--- Your To-Do List ---")
        for i, task in enumerate(tasks):
            status = "[DONE]" if task['completed'] else "[ ]"
            print(f"{i + 1}. {status} {task['name']}")
        print("                        ")

def delete_task(task_number):
    if 1 <= task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
        print(f"Task '{removed_task['name']}' deleted.")
        save_tasks()
    else:
        print("Invalid task number. Please enter a number from the list.")

def mark_task_complete(task_number):
    if 1 <= task_number <=
    len(tasks):
        if not tasks[task_number - 1]['completed']:
            tasks[task_number - 1]['completed'] = True
            print(f"Task '{tasks[task_number - 1]['name']}' marked as
complete.")
            save_tasks()
        else:
            print(f"Task '{tasks[task_number - 1]['name']}' is already
complete.")
    else:
        print("Invalid task number. Please enter a number from the list.")

def get_valid_int_input(prompt):
    while True:
        try:
            value = int(input(prompt))
```

```
            return value
        except ValueError:
            print("Invalid input. Please enter a number.")

def run_todo_list_manager():
    load_tasks() # Load tasks when the manager starts

    while True:
        print("\nTo-Do List Manager")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Delete Task")
        print("4. Mark Task Complete")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            task = input("Enter the task to add: ")
            add_task(task)
        elif choice == '2':
            view_tasks()
        elif choice == '3':
            view_tasks()
            if tasks: # Only ask to delete if there are tasks
                task_num = get_valid_int_input("Enter the number of the
task to delete: ")
                delete_task(task_num)
            else:
                print("No tasks to delete.")
        elif choice == '4':
            view_tasks()
            if tasks: # Only ask to mark complete if there are tasks
                task_num = get_valid_int_input("Enter the number of the
task to mark complete: ")
                mark_task_complete(task_num)
            else:
                print("No tasks to mark complete.")
        elif choice == '5':
            print("Exiting To-Do List Manager. Goodbye!")
```

```
            break
        else:
print("Invalid choice. Please try again.")

# Run the To-Do List Manager
run_todo_list_manager()
```

Output:

```
No existing task file found. Starting with an
empty list.

To-Do List Manager
· Add Task
· View Tasks
· Delete Task
· Mark Task Complete
· Exit
Enter your choice: 1
Enter the task to add: Eggs
Task 'Eggs' added.
Tasks saved to tasks.json.

To-Do List Manager
· Add Task
· View Tasks
· Delete Task
· Mark Task Complete
· Exit
```

```
Enter your choice: 2

--- Your To-Do List ---
1. [ ] Eggs
------------------------

To-Do List Manager
1. Add Task
2. View Tasks
3. Delete Task
4. Mark Task Complete
5. Exit
Enter your choice: 4

--- Your To-Do List ---
1. [ ] Eggs
------------------------
Enter the number of the task to mark complete:
1
Task 'Eggs' marked as complete.
Tasks saved to tasks.json.

To-Do List Manager
1. Add Task
2. View Tasks
3. Delete Task
```

```
· Mark Task Complete
· Exit
Enter your choice: 5
Exiting To-Do List Manager. Goodbye!
```

Comparison:

Original Version	Improved Version
No validation	Input validation added
Tasks stored in memory	Tasks saved to file (JSON)
Cannot mark tasks done	Added "mark complete" feature
No error handling	Added try/except
Simple structure	Modular functions

Task 2 – Version Control Integration

• Initialize a Git repository for your project.

• Perform the following steps:

1. Create and switch to a new branch.
2. Add AI-assisted modifications.
3. Commit with a meaningful message.
4. Merge changes back into the main branch.

- Reflect on how version control supports collaborative development with AI assistance.

Code:

```
def c_to_f(c):
return (c * 9/5) + 32

temp = 30
print(f"{temp}°C in Fahrenheit is: {c_to_f(temp)}°F")
```

Output:

```
30°C in Fahrenheit is: 86.0°F
```

Initialize repo:

```
git init
Initialized empty Git repository in /project/.git/
```

Adding file:

```
git add converter.py
git commit -m "Add temperature converter script"
```

```
[main (root-commit) 4a1e2c1] Add temperature converter script
 1 file changed, 6 insertions(+)
 create mode 100644 converter.py
```

Creating and switching to new branch

```
git checkout -b ai-enhancement
Switched to a new branch 'ai-enhancement'
```

Prompt:

Improve the code:

Code:

```
def c_to_f(c):
    return (c * 9/5) + 32

try:
    temp = float(input("Enter temperature in °C: "))
    print(f"{temp}°C in Fahrenheit is: {c_to_f(temp)}°F")
except ValueError:
    print("Invalid input! Please enter a number.")
```

Output:

```
Enter temperature in °C: 20
20.0°C in Fahrenheit is: 68.0°F

Enter temperature in °C: hello
Invalid input! Please enter a number.
```

Commit:

```
git add converter.py
git commit -m "AI enhancement: Added user input + error
handling"
[ai-enhancement 91f0e22] AI enhancement: Added user input +
error handling
 1 file changed, 10 insertions(+), 2 deletions(-)
```

Merge:

```
git checkout main
git merge ai-enhancement
Switched to branch 'main'
Merge made by the 'recursive' strategy.
 converter.py | 12 ++++++------
```

```
1 file changed, 6 insertions(+), 6 deletions(-)
```

Reflection:

Version control helps maintain a clean development workflow when collaborating with AI.
Using branches allows separation between original and AI-generated enhancements.
Git commits document exactly what AI changed, making it easy to review, revert, or improve further.
Merging combines human-written and AI-suggested improvements without losing work.

Task 3 – AI for Commit Message Generation

- Use AI to generate concise, meaningful commit messages based on the changes made.
- Compare AI-generated messages with human-written ones.
- Discuss how AI can improve documentation and traceability in Git history.

Original code:

```
def c_to_f(c):
    return (c * 9/5) + 32

temp = 30
print(f"{temp}°C in Fahrenheit is: {c_to_f(temp)}°F")
```

Prompt:
Update the code

Modified code:

```
def celsius_to_fahrenheit(c):
```

```
    return (c * 9/5) + 32

def celsius_to_kelvin(c):
    return c + 273.15

temp = 25
print(f"{temp}°C in Fahrenheit:
{celsius_to_fahrenheit(temp)}°F")
print(f"{temp}°C in Kelvin: {celsius_to_kelvin(temp)}K")
```

Commit message:
Added Kelvin conversion function and renamed temperature functions. Updated sample output to include new unit.

Prompt:
Generate a better commit message

Ai commit message:
feat: add Kelvin conversion and improve function naming for clarity

Aspect	Human-Written Message	AI-Generated Message
Length	Longer	Concise
Format	Natural language	Follows conventional commits (`feat:`)
Clarity	Good	Very clear & structured
Traceability	Moderate	High — easy to scan in Git history

Consistency	Depends on user	AI ensures consistency
Professionalism	Varies person to person	More standardized

Task 4 – Pair Programming Simulation

- Student A writes the initial version of a program.
- Student B, with AI assistance, suggests improvements or adds features.
- Both students collaborate using Git branching and merging to integrate their work.

Student A – Initial Program (Simple Calculator)

Prompt:

Generate initial program of add and sum a basic calculator

Student A's initial calculator program

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

print("Calculator")
x = int(input("Enter a: "))
y = int(input("Enter b: "))
```

```
print("Sum =", add(x, y))
print("Difference =", subtract(x, y))
```

Student B + AI Assistance – Improved Version

Prompt:

Improve the code

- Add error handling
- Add multiplication & division
- Validate numeric inputs
- Cleaner UI

AI-assisted improved calculator

```
def get_number(label):
    while True:
        try:
            return float(input(f"Enter {label}: "))
        except ValueError:
            print("Invalid input. Please enter a valid
number.")

def add(a, b): return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    return a / b
```

```
print("=== Enhanced Calculator ===")
x = get_number("number 1")
y = get_number("number 2")

print("Add:", add(x, y))
print("Subtract:", subtract(x, y))
print("Multiply:", multiply(x, y))
print("Divide:", divide(x, y))
```

Initial Version Output

```
Calculator
Enter a: 10
Enter b: 5
Sum = 15
Difference = 5
```

AI-Assisted Version Output

```
=== Enhanced Calculator ===
Enter number 1: 10
Enter number 2: 5
Add: 15.0
Subtract: 5.0
Multiply: 50.0
Divide: 2.0
```

Step 1 — Student A initializes repository

```
git init
git add .
git commit -m "Student A: initial calculator program"
```

Output:

```
[master (root-commit) 1a2b3c] Student A: initial
calculator program
 1 file changed, 12 insertions(+)
```

Step 2 — Student B creates branch

```
git checkout -b ai-improvements
```

Output:

```
Switched to a new branch 'ai-improvements'
```

Step 3 — Student B adds AI-assisted improvements

```
git add calculator.py
git commit -m "Student B: enhanced calculator using AI
suggestions"
```

Output:

```
[ai-improvements 9f8e7d] Student B: enhanced calculator
using AI suggestions
 1 file changed, 32 insertions(+), 4 deletions(-)
```

Step 4 — Merge improved version to main

```
git checkout main
git merge ai-improvements
```

Output:

```
Updating 1a2b3c..9f8e7d
Fast-forward
 calculator.py | 36 ++++++++++++++++++++++++++++-----
```

Side-by-Side Comparison

Feature	Student A Version	Student B (AI-Assisted) Version
Adding numbers	✔	✔
Subtracting	✔	✔
Multiplying	✘	✔
Dividing	✘	✔ (with zero check)
Error Handling	✘	✔
Input Validation	✘	✔

UI	Basic	Polished
Reliability	Low	High

Reflection

This pair programming simulation shows how two students can collaborate efficiently using Git and AI. Student A starts with a simple base program. Student B, with AI assistance, extends the program by adding error handling, extra features, and cleaner structure. Git branching helps both students work independently while keeping the project organized. Merging allows their contributions to combine without conflicts. AI functions like a third collaborator—suggesting improvements, refactoring code, and ensuring best practices. This approach demonstrates how modern development teams use AI and version control to boost productivity and maintain high code quality

---

Task 5 – AI-Assisted Debugging
• Create a program with logical or syntax errors.
• Use AI to identify and fix the errors.
• Compare the buggy and corrected versions.

Prompt:
Can you create a small Python program that has a couple of logical or syntax errors?
Make it simple, like a calculator or average finder.

Bugged code:

```
# Buggy Program: Average Calculator

numbers = [10, 20, 30, 40]

def compute_avg(nums):
    total = 0
    for n in nums:
        total = total + n
```

```
    avg = total / len(num)   # BUG: variable name is
wrong ("num" instead of "nums")
    return avg

print("Average is: " + compute_avg(numbers))    #
BUG:
```

Output:

```
---------------------------------------------------------------------------
NameError                                 Traceback
(most recent call last)

/tmp/ipykernel_2654/3096258958.py in <cell line: 0>()
     10  return avg
     11
---> 12 print("Average is: " + compute_avg(numbers)) #
BUG: concatenating string + float


/tmp/ipykernel_2654/3096258958.py in compute_avg(nums)
      · for n in nums:
      · total = total + n
----> 9  avg = total / len(num)    # BUG: variable
name is wrong ("num" instead of "nums")
     10  return avg
     11


NameError: name 'num' is not defined
```

Prompt:
Here is the code. It has bugs. Can you tell me what is wrong and fix it?

Fixed code:

```
# Fixed Program: Average Calculator

numbers = [10, 20, 30, 40]

def compute_avg(nums):
    total = sum(nums)# More pythonic and correct
    avg = total / len(nums)
    return avg

print(f"Average is: {compute_avg(numbers)}")
```

Output:

```
Average is: 25.0
```

Aspect	Buggy Version	AI-Fixed Version
Wrong variable `num`	✔	✘
String + float error	✔	✘
Uses loop for sum	✔	✘ (uses sum())
Working output	✘	✔

Task 6 – Feature Expansion using AI
• Develop a basic application (e.g., Calculator).
• Use AI to add advanced features (history, validation).
• Analyze improvements.

Prompt:
Can you generate a very basic Python calculator program? Keep it simple with +, - only.

Code:

```
# Basic Calculator (Before AI Improvements)

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

print("Simple Calculator")
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

print("Addition:", add(x, y))
print("Subtraction:", subtract(x, y))
```

Output:

```
Simple Calculator
Enter first number: 10
Enter second number: 5
Addition: 15
Subtraction: 5
```

Prompt:
Now can you expand this calculator by adding input validation and operation history?

Enhanced code:

```
# Enhanced Calculator with Validation + History

history = []

def get_number(label):
    while True:
        try:
            return float(input(f"Enter {label}: "))
        except ValueError:
            print("Invalid number. Try again.")

def add(a, b):
    res = a + b
    history.append(f"Added {a} + {b} = {res}")
    return res

def subtract(a, b):
    res = a - b
    history.append(f"Subtracted {a} - {b} = {res}")
    return res

def multiply(a, b):
    res = a * b
    history.append(f"Multiplied {a} * {b} = {res}")
    return res

def divide(a, b):
    if b == 0:
```

```
        history.append("Attempted division by zero")
        return "Error: Division by zero"
    res = a / b
    history.append(f"Divided {a} / {b} = {res}")
    return res

while True:
    print("\n=== Advanced Calculator ===")
    print("1. Add\n2. Subtract\n3. Multiply\n4. Divide\n5.
View History\n6. Exit")
    choice = input("Choose: ")
    if choice == "6":
        print("Goodbye!")
        break
    elif choice == "5":
        print("\n--- History ---")
        for h in history:
            print(h
        ) continue

    a = get_number("first number")
    b = get_number("second number")

    if choice == "1":
        print("Result:", add(a, b))
    elif choice == "2":
        print("Result:", subtract(a,
        b))
    elif choice == "3":
        print("Result:", multiply(a,
        b))
    elif choice == "4":
        print("Result:", divide(a, b))
```

```
    else:
        print("Invalid option.")
```

Output:

```
=== Advanced Calculator ===
· Add
· Subtract
· Multiply
· Divide
· View History
· Exit
Choose: 1
Enter first number: 10
Enter second number: 5
Result: 15

=== Advanced Calculator ===
Choose: 5

--- History ---
Added 10.0 + 5.0 = 15.0
```

Feature	Basic Version	AI-Enhanced Version
Adds & subtracts	✔	✔
Multiply/Divide	✘	✔
Input validation	✘	✔
Handles errors	✘	✔

Operation history	✘	✔
User menu	Basic	Advanced
Crashes on wrong input	✔	✘
End-user friendly	Low	High

Analysis:
The basic calculator was functional but limited—it only performed addition and subtraction and would crash if users entered invalid input. The AI-assisted version significantly improved the program by adding input validation, new operations (multiply & divide), and a persistent history log of all calculations. These enhancements increase usability, reduce errors, and make the application more user-friendly. This demonstrates how AI can help expand features, prevent bugs, and speed up development through collaborative refinement.